

FACULDADE NOVA ROMA
CURSO DE CIÊNCIA DA COMPUTAÇÃO

JOSÉ AILTON BEZERRA JUNIOR

**A ESTRATÉGIA DE PLATAFORMA DE APIS: DE PADRÕES
TÉCNICOS A PRODUTOS DIGITAIS**

RECIFE
2026

JOSÉ AILTON BEZERRA JUNIOR

**A ESTRATÉGIA DE PLATAFORMA DE APIS: DE PADRÕES
TÉCNICOS A PRODUTOS DIGITAIS**

Trabalho de Conclusão de Curso
apresentado ao Curso de Ciência da
Computação da Faculdade Nova Roma,
como requisito parcial para a obtenção
do título de Bacharel em Ciência da
Computação.

Orientador: Prof. Henning Summer

RECIFE

2026

JOSÉ AILTON BEZERRA JUNIOR

**A ESTRATÉGIA DE PLATAFORMA DE APIS: DE PADRÕES TÉCNICOS A
PRODUTOS DIGITAIS**

Trabalho de Conclusão de Curso apresentado ao Curso de Ciência da Computação da Faculdade Nova Roma, como requisito parcial para a obtenção do título de Bacharel em Ciência da Computação.

Aprovado em: 06/ 07/ 2026

BANCA EXAMINADORA

Prof. MSc. Henning Summer (Orientador)

Faculdade Nova Roma

Prof. MSc. Davi Maia

Faculdade Nova Roma

Prof. Marcelo

Faculdade Nova Roma

Dedico este trabalho a Deus, por todas as bênçãos e oportunidades; à minha falecida avó, que sempre torceu para que eu me formasse; e aos meus pais, por todo o sacrifício, apoio diário, paciência e incentivo constante ao longo desta jornada acadêmica e profissional.

AGRADECIMENTOS

Agradeço, primeiramente, a Deus, por me conceder força, sabedoria e perseverança para superar os desafios e concluir mais esta importante etapa da minha vida.

Aos meus pais Ailton e Rosângela, em especial à minha avó, pelo amor, suporte incondicional e por sempre acreditarem no meu potencial desde o início desta jornada.

À minha irmã, Roseane, que sempre esteve ao meu lado, apoiando-me antes, durante e após cada etapa da minha vida, não sendo diferente durante toda a trajetória na faculdade. Seu incentivo e carinho foram fundamentais para que eu chegasse até aqui.

À minha namorada, Kailane Maria, por toda a força, apoio, compreensão e motivação ao longo dessa caminhada. Seu incentivo constante foi essencial para que eu mantivesse o foco e a determinação até a conclusão deste curso.

Ao meu cunhado, Adilson, por ter sido a pessoa que me incentivou e indicou o caminho para iniciar esta graduação. Hoje, poder celebrar esta conquista e concluir o curso também é resultado daquele primeiro incentivo, pelo qual sou profundamente grato.

Ao meu orientador, Prof. Henning Summer, pela condução, paciência e pelas valiosas orientações técnicas e acadêmicas, fundamentais para a construção deste trabalho.

Aos professores do Curso de Ciência da Computação da Faculdade Nova Roma, pelos conhecimentos compartilhados ao longo desses anos, que contribuíram significativamente para minha formação acadêmica e profissional.

Por fim, agradeço aos colegas de profissão e a todos que, de alguma forma, contribuíram para o meu crescimento como desenvolvedor e para a realização desta conquista.

*“Qualquer tolo consegue escrever código que
um computador entenda. Bons
programadores escrevem código que
humanos entendam.”*

— Martin Fowler (Refactoring, 2018)

RESUMO

Sistemas corporativos legados representam um desafio recorrente para organizações que precisam evoluir sua infraestrutura sem comprometer a estabilidade do que já está em operação. Em muitos casos, aplicações modernas conectam-se diretamente a bancos de dados relacionais que concentram décadas de regras de negócio, o que amplia o acoplamento e cria condições para falhas de integração e exposição indevida de dados — preocupação ainda mais relevante diante de normativas como a Lei Geral de Proteção de Dados (LGPD). Este trabalho tem por objetivo demonstrar a viabilidade de uma camada intermediária governada, estruturada sobre dois pilares: a abordagem API-First, que define o contrato antes da implementação, e a Governança como Código (Policy-as-Code), que automatiza a validação de regras de qualidade e segurança. Como método, adotou-se a pesquisa aplicada conduzida por meio de uma prova de conceito, ambientada em uma clínica médica fictícia, na qual uma regra de faturamento residente no PostgreSQL passa a ser acessada exclusivamente por meio de uma camada intermediária em Node.js e TypeScript. O contrato foi formalizado em OpenAPI 3.0.3 e validado pelo linter Spectral, integrado ao processo de build. Os resultados, obtidos a partir de evidências de execução reproduzíveis, mostram que a camada intermediária atua como barreira de proteção: a validação automatizada do contrato reportou zero erros, o mascaramento de CPF foi confirmado em requisição HTTP real (retornando o dado no padrão `***.***.***-XX`), o acesso sem autenticação foi rejeitado com HTTP 401, e o consumidor em React comunica-se apenas com a API, sem qualquer driver de banco. Conclui-se que a Engenharia de Plataforma, mais do que uma estratégia técnica de desacoplamento, constitui um caminho viável para transformar integrações frágeis em serviços digitais mais seguros, governados e sustentáveis ao longo do tempo.

Palavras-chave: API-First. Camada Anticorrupção. Governança como Código. Sistemas legados. LGPD.

ABSTRACT

Legacy enterprise systems pose a recurring challenge for organizations that need to evolve their infrastructure without compromising the stability of what is already in production. In many cases, modern applications connect directly to relational databases that concentrate decades of business rules, increasing coupling and creating conditions for integration failures and undue data exposure — a concern that becomes even more relevant under regulations such as the Brazilian General Data Protection Law (LGPD). This work aims to demonstrate the feasibility of a governed intermediary layer, structured on two pillars: the API-First approach, which defines the contract before implementation, and Policy-as-Code, which automates the validation of quality and security rules. Methodologically, applied research was conducted through a proof of concept set in a fictitious medical clinic, in which a billing rule residing in PostgreSQL becomes accessible exclusively through an intermediary layer built with Node.js and TypeScript. The contract was formalized in OpenAPI 3.0.3 and validated by the Spectral linter, integrated into the build process. The results, obtained from reproducible execution evidence, show that the intermediary layer acts as a protection barrier: automated contract validation reported zero errors, CPF masking was confirmed in a real HTTP request (returning the value in the `***.***.***-XX` pattern), unauthenticated access was rejected with HTTP 401, and the React consumer communicates only with the API, without any database driver. We conclude that Platform Engineering, more than a technical decoupling strategy, is a viable path to turn fragile integrations into safer, governed, and sustainable digital services over time.

Keywords: API-First. Anti-Corruption Layer. Policy-as-Code. Legacy systems. LGPD.

SUMÁRIO

1 INTRODUÇÃO.....;	11
1.1 Contexto e problema.....	13
1.2 Justificativa.....	14
1.3 Objetivos.....	15
2 FUNDAMENTAÇÃO TEÓRICA.....	16
2.1 Evolução das arquiteturas e o peso do banco de dados legado.....	16
2.2 A Camada Anticorrupção como estratégia de isolamento.....	17
2.3 API-First e contratos com OpenAPI.....;	18
2.4 Governança como Código, qualidade e conformidade.....	19
2.5 Trabalhos relacionados.....	21
2.6 Engenharia de Plataforma e a API como produto.....	22
3 METODOLOGIA.....	23
3.1 Caracterização da pesquisa.....	23
3.2 O ciclo de Design Science Research.....	24
3.3 Arquitetura proposta e escolha tecnológica.....	25
3.4 Processo de implementação e decisões arquiteturais.....	27
3.5 Critérios de verificação da prova de conceito.....	28
3.6 Estrutura do repositório e reprodutibilidade.....	29
4 DESENVOLVIMENTO DA PROVA DE CONCEITO.....	29
4.1 Camada de dados legada: regras e riscos estruturais.....	30
4.2 Integração do contrato OpenAPI ao build.....	31
4.3 Padronização do tratamento de erros.....	31
4.4 Criação da camada repository.....	32
4.5 Validação de payload com Zod.....	33
4.6 Autenticação JWT e controle de acesso por papel.....	33
4.7 Testes unitários com Jest.....	34
4.8 Pipeline de integração contínua (GitHub Actions).....	35
4.9 Health checks: liveness e readiness.....	35
4.10 Visão consolidada da arquitetura final.....	36

5 RESULTADOS E DISCUSSÃO.....	37
5.1 Governança de contrato e bloqueio de não conformidades.....	37
5.2 Tratamento de dados sensíveis na camada anticorrupção.....	38
5.3 Qualidade verificável por testes automatizados.....	39
5.4 Desacoplamento técnico do consumidor em relação ao legado	40
5.5 Síntese dos resultados e discussão.....	40
 6 CONCLUSÃO.....	 42
6.1 Resposta à questão de pesquisa.....	42
6.2 Limitações do trabalho.....	43
6.3 Contribuições do trabalho.....	45
6.4 Sugestões para trabalhos futuros.....	45
6.5 Consideração final.....	46
 7 REFERÊNCIAS.....	 48

1 INTRODUÇÃO

Poucos problemas da engenharia de software são tão persistentes quanto o dos sistemas legados. Bennett e Rajlich (2000) os definem de maneira deliberadamente informal, como sistemas grandes que não sabemos bem como tratar, mas dos quais a organização depende para funcionar. A imprecisão é proposital. Um sistema raramente se torna legado por uma razão técnica isolada: ele se torna legado quando o custo de mudá-lo passa a crescer mais rápido do que o risco de deixá-lo como está, e mesmo assim ele continua no ar, sustentando operações críticas.

Assunção et al. (2025) mostram que essa condição não é residual. Ao mapear uma década de pesquisa sobre modernização de software, os autores identificam que a necessidade de lidar com sistemas antigos permanece sendo um dos motores centrais da atividade de engenharia, impulsionada por redução de custo operacional, ganho de desempenho e diminuição de complexidade. Modernizar, nesse sentido, não é um projeto ocasional, é parte do trabalho corrente.

Em setores críticos como a saúde, o cenário costuma ter uma característica adicional que agrava o quadro. As regras de negócio não vivem apenas no código da aplicação, elas migraram, ao longo de anos, para dentro do próprio banco de dados relacional. Cálculos de faturamento, validações de convênio e consolidações de atendimento acabam escritos em stored procedures, views e funções, com destaque para linguagens como o PL/pgSQL. A decisão fez sentido quando o banco era pensado como núcleo transacional da aplicação, e hoje cobra o seu preço.

O preço aparece na hora de evoluir. Renomear uma coluna, ajustar uma view ou alterar um tipo de dado deixa de ser uma operação local e passa a produzir efeitos em pontos distantes do ecossistema, muitos deles desconhecidos. Ao mesmo tempo, a pressão por consumo moderno não para de crescer: portais web, aplicativos móveis, painéis administrativos e

integrações com parceiros precisam daqueles mesmos dados. A saída mais rápida costuma ser expor o legado por meio de APIs.

O problema é o que acontece quando essas APIs nascem sem processo. A OWASP (2023) classifica esse fenômeno como Improper Inventory Management, o nono risco mais crítico da sua lista de segurança de APIs, e o associa diretamente à figura das Shadow APIs: endpoints que existem, respondem e acessam a mesma fonte de dados da API oficial, mas estão fora de qualquer catálogo, documentação ou controle. Não são invenções de laboratório. Segundo a própria OWASP, decorrem de documentação ausente ou desatualizada e de inventários incompletos, condições que qualquer equipe sob pressão de prazo reproduz sem perceber.

Este trabalho parte de um protótipo que ilustra esse contexto com fidelidade incômoda. Nele, regras de negócio residem no PostgreSQL por meio de funções em PL/pgSQL, como a função `gerar_resumo_faturamento`, e são expostas por uma API em Express com TypeScript, com separação entre as camadas de controle e de serviço, evitando o acesso direto ao banco pelos consumidores. O contrato segue a abordagem API-First, formalizado em um arquivo OpenAPI 3.0.3 e verificado pelo linter Spectral. No papel, portanto, a governança existia.

Na prática, não. A verificação do contrato rodava separada da compilação, de modo que o código avançava mesmo com a especificação fora de conformidade, e o mecanismo de autenticação, embora declarado no esquema via `bearerAuth`, não estava implementado no backend. A distância entre o contrato desejado e a governança efetivamente praticada é, provavelmente, a tensão mais comum no ciclo de vida de APIs, e era ela que o protótipo materializava.

Diante disso, este trabalho propõe uma abordagem de modernização incremental apoiada em API-First e em Governança como Código (Policy-as-Code), na qual a API atua como camada intermediária governada entre as aplicações modernas e o sistema legado. A intenção não é criar endpoints, e sim estabelecer um modelo replicável no qual contratos, padrões e políticas sejam verificáveis ao longo do ciclo de vida do sistema, reduzindo o

acoplamento com o legado e mitigando riscos de exposição indevida de dados, questão especialmente sensível em contextos que exigem conformidade com a Lei Geral de Proteção de Dados (BRASIL, 2018).

1.1 Contexto e problema

Quando regras críticas de negócio estão encapsuladas no banco de dados, o consumidor herda uma dependência que não escolheu. Ele passa a depender do esquema, da nomenclatura interna e do comportamento das rotinas, e não apenas do resultado que precisa. Duas práticas se repetem nesse cenário, e ambas custam caro.

A primeira é o acesso direto ao banco por clientes externos. Amplia o acoplamento e aumenta a superfície de ataque, já que cada consumidor precisa de credenciais e de conhecimento sobre a estrutura interna. A segunda é a criação acelerada de endpoints ad hoc, que expõem consultas e funções sem padronização, com documentação frágil e controles de segurança aplicados de forma irregular. É exatamente o caminho que leva às Shadow APIs descritas pela OWASP (2023).

O efeito técnico é conhecido. Fowler (2002) já argumentava que misturar lógica de domínio com acesso a dados produz sistemas difíceis de testar e de evoluir, e propunha a separação por meio de camadas dedicadas de mapeamento e de repositório. Quando essa separação não existe, qualquer mudança no banco vira, potencialmente, uma quebra de contrato para quem consome.

O efeito de conformidade é menos discutido, mas não é menor. Em ambientes de saúde, os dados tratados são pessoais e frequentemente sensíveis. Sem mecanismos sistemáticos de autorização e de minimização, o risco de exposição cresce, e, sem governança explícita, a organização perde a capacidade de demonstrar o que expõe, para quem, com qual finalidade e sob quais garantias. A LGPD (BRASIL, 2018) trata essa demonstrabilidade como obrigação, não como boa prática.

O problema central, portanto, não é disponibilizar dados via API. É construir uma integração governada, na qual o contrato seja consistente e

verificado automaticamente, e na qual políticas mínimas de segurança, padronização e qualidade impeçam que a evolução aconteça no improviso.

Formalmente, a questão de pesquisa pode ser enunciada assim: *de que maneira uma camada intermediária baseada em API-First e Policy-as-Code pode reduzir o acoplamento entre clientes modernos e um sistema legado com regras encapsuladas no banco, elevando a padronização do contrato e o controle sobre a evolução da API, com ênfase em segurança e confiabilidade no tratamento de dados sensíveis?*

1.2 Justificativa

A relevância deste trabalho se apoia em duas frentes que se sustentam mutuamente.

Do ponto de vista técnico, reescrever um sistema legado do zero costuma ser inviável. Seacord, Plakosh e Lewis (2003) descrevem a substituição total como a estratégia de maior risco entre as disponíveis, justamente porque exige que a organização reproduza, de uma só vez, décadas de conhecimento tácito acumulado no sistema em operação. Assunção et al. (2025) chegam a conclusão semelhante ao revisar a literatura recente, apontando que abordagens incrementais predominam nos estudos justamente por permitirem entrega de valor durante a transição.

Nesse contexto, uma camada intermediária que se aproxima do padrão Camada Anticorrupção (Anti-Corruption Layer), descrito por Evans (2003), representa uma estratégia mais realista. Ela permite que clientes modernos evoluam com menor dependência do banco, sem exigir a substituição imediata da infraestrutura existente. No projeto analisado, essa separação já aparece na prática, por meio da divisão entre controller e service e do consumo das funções do banco por uma camada de repositório, no sentido proposto por Fowler (2002).

Do ponto de vista organizacional e de conformidade, a ausência de controles mínimos em ambientes com dados sensíveis compromete tanto a segurança quanto a adequação regulatória. Processos baseados em revisão manual não escalam à medida que o número de APIs cresce, e a OWASP

(2023) registra que a defasagem entre a documentação declarada e o que efetivamente roda em produção é uma das causas mais frequentes de incidente.

Automatizar políticas é a resposta que a literatura técnica tem oferecido para esse descompasso. Chandramouli (2022), em publicação técnica do NIST, recomenda que verificações de segurança e conformidade sejam integradas diretamente às esteiras de integração e entrega contínuas, deslocando o controle para o início do ciclo. Verificar o contrato OpenAPI, os padrões de resposta, as convenções de nomenclatura e os requisitos mínimos de segurança dentro do próprio build agrega valor concreto, tanto acadêmico quanto prático, e é o que este trabalho se propõe a demonstrar.

1.3 Objetivos

O objetivo geral deste trabalho é demonstrar a viabilidade de uma camada intermediária baseada em API-First e Governança como Código (Policy-as-Code) capaz de reduzir o acoplamento entre clientes modernos e um sistema legado com regras encapsuladas no banco (PL/pgSQL), elevando a padronização do contrato e o controle sobre a evolução da API, com ênfase em segurança e confiabilidade no contexto de dados sensíveis.

Para atingir esse objetivo geral, foram definidos os seguintes objetivos específicos:

- (i) implementar um banco PostgreSQL representando o legado, com regras de negócio em funções e procedimentos PL/pgSQL, incluindo a consolidação de faturamento;
- (ii) definir o contrato da API antes da implementação, por meio de uma especificação OpenAPI 3.0.3, adotando a abordagem API-First;
- (iii) aplicar verificações automatizadas do contrato com o Spectral, garantindo consistência e aderência a políticas de qualidade;
- (iv) desenvolver a API intermediária em Express com TypeScript, organizada em camadas de controller, service e repository, consumindo as funções do banco e controlando a exposição de dados segundo o princípio da mínima exposição;

(v) verificar a abordagem por meio de critérios técnico-funcionais explícitos, evidenciando o desacoplamento do cliente em relação ao banco e a governança automatizada do contrato como parte integrante do processo de entrega, reduzindo o espaço para o surgimento de Shadow APIs.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo reúne os fundamentos que sustentam a proposta do trabalho. Trata primeiro de como o acoplamento ao banco de dados fragiliza um sistema e trava sua evolução, depois de como o padrão Camada Anticorrupção reduz esse impacto ao isolar modelos e contratos, em seguida de como a abordagem API-First com OpenAPI fortalece o alinhamento entre equipes, e de como a Governança como Código automatiza qualidade, segurança e conformidade. Uma seção de trabalhos relacionados situa este projeto frente a propostas semelhantes na literatura, e o capítulo se fecha com a ideia de Engenharia de Plataforma, que amarra os pilares anteriores em um conceito só.

2.1 Evolução das arquiteturas e o peso do banco de dados legado

Sistemas legados costumam ter nascido em um período no qual as arquiteturas monolíticas eram a escolha natural. Interface, regras de negócio e acesso a dados evoluíam como uma unidade só, com alta dependência entre as partes. Com o tempo, ficou comum transferir responsabilidades críticas da aplicação para o banco relacional, consolidando lógica em stored procedures, views e triggers. Bennett e Rajlich (2000) descrevem esse processo como parte natural da evolução de qualquer sistema útil: todo software bem-sucedido atrai pedidos de mudança, e cada mudança tende a se acomodar onde for mais barato implementá-la naquele momento, ainda que isso custe caro depois.

Quando regras importantes residem no banco, a aplicação passa a estruturar seu raciocínio conforme a forma como os dados estão modelados internamente. Mudanças aparentemente simples, como renomear colunas, alterar tipos de dados ou reorganizar tabelas, podem provocar falhas em cadeia. Em sistemas de saúde, rotinas de relatório e faturamento, cálculos de agregação e cruzamento de informações entre paciente, convênio, atendimento e cobrança acabam encapsulados em funções e procedimentos. O banco deixa de ser apenas um mecanismo de persistência e passa a carregar lógica de negócio, geralmente pouco observável e difícil de versionar com o mesmo rigor aplicado ao código da aplicação.

O impacto fica mais evidente quando clientes modernos, como aplicações web ou móveis, tentam consumir o legado por meio de APIs. Sem uma camada de proteção, a API tende a espelhar o banco: nomes de campos, formatos e estruturas internas vazam para o contrato público. O resultado é uma evolução lenta e arriscada, na qual cada ajuste no banco ameaça quebrar integrações e exige coordenação intensa entre equipes. Em síntese, o banco legado se transforma em um ponto de rigidez, um componente crítico e difícil de modificar, que limita escalabilidade, manutenção e velocidade de entrega.

2.2 A Camada Anticorrupção como estratégia de isolamento

Diante do desafio de evoluir sem reescrever o legado, padrões de arquitetura propõem mecanismos de isolamento entre domínios distintos. Um dos mais relevantes nesse contexto é a Camada Anticorrupção (Anti-Corruption Layer, ACL), associada ao Domain-Driven Design e descrita por Evans (2003). A ACL funciona como uma barreira de tradução. Seu propósito é impedir que o modelo legado contamine o modelo moderno e, ao mesmo tempo, evitar que decisões do mundo moderno imponham mudanças diretas ao legado.

A modernização de sistemas legados monolíticos impõe o desafio de mitigar riscos de falha durante a transição. Nesse contexto, a ACL pode atuar não só como tradutora de modelos de domínio, mas também como base prática para a implementação do padrão Estrangulador (Strangler Fig), descrito por Fowler (2004), que permite uma substituição gradual e segura do sistema, transformando projetos monolíticos de alto risco em programas de

modernização iterativos com entrega contínua de valor. Essa abordagem incremental também é recomendada na literatura sobre decomposição de monólitos em microsserviços (RICHARDSON, 2018), que descreve a ACL e o padrão Estrangulador como mecanismos centrais para migrar funcionalidades de forma controlada, sem interromper o sistema em operação. Estudos sobre modernização de sistemas legados para arquiteturas de microsserviços (ALMEIDA et al., 2024; WOLFART et al., 2021) reforçam que estratégias incrementais reduzem os riscos associados à transformação, ao garantir maior controle sobre a interação entre os sistemas envolvidos.

Ao inserir a API intermediária, a organização reduz o risco de que inconsistências e o acoplamento excessivo do banco legado se propaguem para os componentes modernos, isolando os domínios e permitindo que novas funcionalidades evoluam de maneira mais independente. Na prática, a ACL define uma camada responsável por traduzir contratos e modelos do formato legado para formatos mais estáveis e orientados ao consumo, encapsular detalhes internos do banco, como estrutura, nomenclatura e particularidades de consulta, centralizar regras de exposição, incluindo normalização, formatação, mascaramento e tratamento de erros, e reduzir o impacto de mudanças, de modo que, se o banco for alterado, apenas a tradução precise ser ajustada.

No contexto deste projeto, a API em Express com TypeScript cumpre o papel de ACL ao intermediar o acesso a regras implementadas no banco, expondo-as como endpoints REST com respostas padronizadas. Isso é especialmente importante quando a lógica reside em stored procedures e é consumida por aplicações modernas: a ACL impede que o cliente precise conhecer convenções internas e estruturas legadas. É nessa camada, também, que políticas de proteção de dados, como não expor identificadores sensíveis em formato bruto, devem ser aplicadas, tratando segurança e privacidade como parte do desenho arquitetural, e não como correção pontual feita depois do fato.

2.3 API-First e contratos com OpenAPI

Uma camada intermediária só é realmente eficaz quando o contrato que ela expõe é claro, estável e verificável. Historicamente, muitas equipes

seguiram uma abordagem Code-First, na qual a API é implementada primeiro e documentada depois. Esse fluxo, embora comum, costuma produzir documentação incompleta, que diverge do comportamento real e é difícil de manter.

A abordagem API-First inverte essa lógica: o contrato deve ser definido e verificado antes da implementação. Significa desenhar rotas, parâmetros, respostas, erros e mecanismos de segurança de forma prévia e estruturada. Nesse ponto, a especificação OpenAPI (OPENAPI INITIATIVE, 2021) assume papel central. Ao descrever a API de forma padronizada, ela passa a funcionar como fonte única de verdade para a equipe, tanto para quem implementa quanto para quem consome. Os principais ganhos dessa abordagem, no contexto de modernização com ACL, são o alinhamento antecipado entre equipes, a previsibilidade dos contratos, a evolução controlada via versionamento e a redução de retrabalho decorrente de incompatibilidades descobertas tardiamente.

No projeto analisado, o contrato já existe na forma de um arquivo OpenAPI, o `swagger.yaml`. O passo arquitetural, e também acadêmico, foi consolidar esse artefato como parte ativa do processo: não um arquivo para abrir manualmente quando necessário, mas um documento governado, verificado e integrado à rotina de desenvolvimento.

2.4 Governança como Código, qualidade e conformidade

À medida que as APIs crescem em quantidade e criticidade, manter padrões de segurança, nomenclatura e consistência só por meio de revisão manual se torna inviável. Surge, então, a abordagem de Governança como Código (Policy-as-Code): em vez de depender de regras informais ou da atenção individual dos revisores, as políticas passam a ser expressas como verificações automatizadas que rodam no fluxo de desenvolvimento, por exemplo em pipelines de integração contínua.

A verificação automatizada de contratos de API, operacionalizada por ferramentas como o Spectral (STOPLIGHT, 2021), materializa o conceito de Policy-as-Code. Em arquiteturas corporativas modernas, essa prática é

importante para a implantação do modelo DevSecOps, no qual os controles de segurança e conformidade são deslocados para o início do ciclo de desenvolvimento, abordagem conhecida como shift-left. As diretrizes técnicas do NIST (CHANDRAMOULI, 2022) para aplicações baseadas em microsserviços recomendam a integração de verificações automatizadas diretamente nas esteiras de integração e entrega contínuas, para mitigar anomalias de configuração e favorecer estados seguros em tempo de execução. Ao se tornarem executáveis, as regras de governança bloqueiam proativamente o avanço de contratos fora do padrão e reduzem o espaço para o surgimento de endpoints não rastreados, as Shadow APIs discutidas na Introdução (OWASP, 2023).

Nessa abordagem, o contrato OpenAPI deixa de ser apenas documentação e passa a ser um artefato testável. Ferramentas como o Spectral permitem definir e aplicar regras diretamente sobre a especificação, garantindo, entre outros aspectos, a presença de descrições mínimas, a padronização de respostas de sucesso e de erro, a consistência de nomenclatura, a exigência de esquemas para requisição e resposta, a declaração coerente de requisitos de segurança e o bloqueio de exposição indevida de campos sensíveis. O benefício direto é que falhas de design, inconsistências e riscos de segurança são identificados antes do deploy: o pipeline atua como guardião de qualidade, e contratos que violam políticas têm o avanço bloqueado automaticamente.

No contexto de saúde e da LGPD, essa automação é ainda mais relevante. Dados de pacientes e identificadores associados são sensíveis por natureza e exigem proteção sistemática. Políticas como a proibição de retornar o CPF em formato bruto ou a obrigatoriedade de autenticação em endpoints críticos precisam ser tratadas como requisitos arquiteturais verificáveis, e não como recomendações. Cabe aqui uma distinção técnica importante. A LGPD (BRASIL, 2018), em seu art. 13, parágrafo único, define a pseudonimização como o tratamento por meio do qual um dado perde a possibilidade de associação a um indivíduo, exceto pelo uso de informação adicional mantida separadamente em ambiente controlado, já o art. 12 trata da anonimização, processo após o qual o dado deixa de ser considerado dado pessoal. O

masking de CPF implementado neste trabalho se enquadra na primeira categoria, a pseudonimização, pois o dado original permanece no banco e o controlador mantém a capacidade de reidentificação. Assim, a combinação de ACL, API-First e Policy-as-Code forma um conjunto coerente: a ACL controla a exposição, o OpenAPI define o contrato, e a Governança como Código garante que esse contrato permaneça consistente, seguro e auditável ao longo de toda a evolução do sistema.

2.5 Trabalhos relacionados

A literatura sobre modernização de sistemas legados costuma se concentrar em três frentes, que raramente aparecem combinadas em um único estudo: a estratégia de migração em si, os padrões arquiteturais de isolamento e a governança do artefato resultante. Esta seção situa o presente trabalho frente a essas três frentes.

Wolfart et al. (2021) propõem um roteiro (roadmap) para modernizar sistemas legados em direção a microsserviços, com ênfase na decomposição incremental e na redução de risco durante a transição. O roteiro reforça o valor de estratégias graduais, mas concentra-se na decisão de quando e como decompor o monólito, sem detalhar como o contrato entre o novo componente e o consumidor deve ser governado ao longo do tempo. Almeida et al. (2024) chegam a um diagnóstico semelhante em um estudo terciário sobre modernização para microsserviços: identificam a ACL e o padrão Estrangulador como mecanismos centrais amplamente citados na literatura, mas observam que grande parte dos estudos não formaliza o contrato exposto por essas camadas, tratando-o como detalhe de implementação em vez de artefato de primeira classe.

Richardson (2018) descreve a ACL como parte do repertório de padrões para decompor monólitos, situando-a ao lado do padrão Estrangulador como mecanismo de migração controlada. O foco do autor está na decomposição em si, não na verificação contínua do contrato que a camada expõe, lacuna que também aparece nos dois trabalhos anteriores.

É justamente nesse ponto que a literatura sobre governança de API contribui. A OWASP (2023) documenta como a ausência de inventário e de verificação formal do contrato produz Shadow APIs, e Chandramouli (2022), em publicação técnica do NIST, recomenda a automação dessas verificações dentro do pipeline de integração contínua, prática que a literatura de DevSecOps chama de shift-left. Nenhum dos dois trabalhos, porém, discute modernização de sistemas legados especificamente, tratam de API governance em termos gerais, aplicáveis a qualquer contexto de desenvolvimento.

A lacuna que este trabalho busca preencher está na interseção dessas duas frentes. Diferente de Wolfart et al. (2021) e de Almeida et al. (2024), que tratam a camada anticorrupção como decisão arquitetural sem aprofundar sua governança contínua, e diferente da literatura de API governance (OWASP, 2023; CHANDRAMOULI, 2022), que não é ambientada em um cenário de modernização de legado, este trabalho integra os dois planos: usa a ACL como estratégia de isolamento entre o legado e o consumidor moderno, e usa Policy-as-Code, operacionalizado por meio do Spectral, para tornar o contrato dessa camada um artefato verificável e sustentável ao longo da evolução do sistema. A contribuição não está em nenhum dos dois pilares isoladamente, ambos amplamente documentados, e sim na demonstração de que eles funcionam de forma integrada em um cenário legado real, ainda que o teste dessa integração aqui se limite a uma prova de conceito em ambiente controlado.

2.6 Engenharia de Plataforma e a API como produto

Os três pilares discutidos até aqui, Camada Anticorrupção, API-First e Governança como Código, convergem para um conceito mais amplo, que organiza a forma como serviços internos são oferecidos e consumidos: a Engenharia de Plataforma (Platform Engineering). Skelton e Pais (2019) descrevem a plataforma como um agrupamento de serviços, padrões e ferramentas oferecidos como serviço (X-as-a-Service) a equipes consumidoras, de modo a reduzir a carga cognitiva de quem desenvolve aplicações e a acelerar a entrega de valor. Nessa perspectiva, a API deixa de ser um detalhe de implementação e passa a ser tratada como um produto: possui contrato

explícito, consumidores bem definidos, requisitos de qualidade e um ciclo de vida governado.

Aplicada ao problema deste trabalho, essa visão conecta as decisões técnicas a um objetivo organizacional. A camada intermediária não é apenas uma barreira de isolamento, a ACL, nem apenas um contrato verificado, o API-First com Policy-as-Code. Ela é um produto digital governado, cuja interface estável protege os consumidores das mudanças do legado e cujas políticas de segurança e padronização são oferecidas de forma transparente. É essa transição, de integrações improvisadas para produtos digitais com contrato, governança e observabilidade, que caracteriza a passagem de padrões técnicos a produtos digitais anunciada no título.

3 METODOLOGIA

Este capítulo descreve o método adotado para construir e avaliar a prova de conceito de uma plataforma de APIs governada, aplicada a um cenário com sistema legado, com o objetivo de reduzir o acoplamento técnico e mitigar a exposição de dados sensíveis. A metodologia está organizada em quatro eixos: a caracterização da pesquisa, o mapeamento do ciclo de Design Science Research seguido no trabalho, a descrição da arquitetura e do processo de governança adotado, e os critérios de verificação e as evidências de execução coletadas.

A hipótese de trabalho pode ser assim enunciada: a aplicação de um fluxo API-First com verificação automática de contrato, Policy-as-Code, de uma camada anticorrupção entre consumidores e o legado, e de mecanismos automatizados de segurança e qualidade contribui para reduzir o acoplamento técnico e diminuir o risco de exposição de dados sensíveis no consumo de APIs. Registra-se que, por se tratar de uma prova de conceito em cenário controlado, sem grupo de comparação ou baseline quantitativo, o trabalho busca demonstrar a viabilidade e o funcionamento dos mecanismos propostos, e não estabelecer uma medida estatística de redução de acoplamento.

3.1 Caracterização da pesquisa

A pesquisa é de natureza aplicada, pois busca resolver um problema concreto observado em integrações com sistemas legados: alto acoplamento, dificuldade de evolução e risco de exposição indevida de dados sensíveis durante a comunicação entre sistemas. Quanto ao delineamento, adota-se uma abordagem orientada a projeto, o Design Science Research (DSR), conforme proposto por Hevner et al. (2004) e operacionalizado segundo o processo de seis atividades de Peffers et al. (2007), na qual o conhecimento sobre um problema e sua solução é obtido por meio da construção e da avaliação de um artefato. Aqui, o artefato, a plataforma de APIs governada, é construído e avaliado em um cenário controlado que simula o ecossistema de uma clínica médica com legado.

A unidade de análise é o fluxo de integração entre três elementos: o legado, representado por um banco relacional PostgreSQL com regras internas em PL/pgSQL, a camada de serviço intermediária, que encapsula o legado, e um consumidor moderno em React. O foco da avaliação é verificar, de forma reproduzível, se a governança de contrato e a camada anticorrupção são capazes de impedir não conformidades e de padronizar e filtrar dados antes de seu consumo.

3.2 O ciclo de Design Science Research aplicado ao trabalho

Peffers et al. (2007) organizam a pesquisa em design em seis atividades: identificação do problema e motivação, definição dos objetivos de uma solução, projeto e desenvolvimento, demonstração, avaliação e comunicação. O Quadro 1 mapeia cada uma dessas atividades às seções deste trabalho em que ela foi conduzida, tornando explícito o ciclo percorrido na construção do artefato.

Quadro 1 — Ciclo de Design Science Research aplicado ao trabalho.

Atividade (Peffers et al., 2007)	O que foi feito	Onde está registrado
1. Identificação do problema e motivação	Diagnóstico do acoplamento entre consumidores modernos e o	Seção 1.1, Contexto e problema

	legado em PL/pgSQL, e do risco de Shadow APIs sem governança.	
2. Definição dos objetivos da solução	Formulação do objetivo geral e dos cinco objetivos específicos que orientam o artefato.	Seção 1.3, Objetivos
3. Projeto e desenvolvimento	Construção da camada intermediária em oito passos incrementais, cada um com decisão arquitetural registrada.	Capítulo 4, Desenvolvimento da prova de conceito
4. Demonstração	Execução do artefato em ambiente local, com o banco populado e o backend em operação, coletando evidências reais de requisição e resposta.	Seções 5.1 a 5.4, evidências de execução
5. Avaliação	Verificação do artefato frente aos quatro critérios definidos na Seção 3.5, com discussão dos limites de cada evidência.	Seção 5.5, Síntese dos resultados
6. Comunicação	Registro das decisões arquiteturais em quadros, documentação do repositório e este próprio texto e a uma publicação futura.	Capítulo 6, Conclusão

A leitura do Quadro 1 evidencia que o ciclo não foi percorrido uma única vez de forma linear. As atividades de projeto e desenvolvimento e de demonstração se alternaram ao longo dos oito passos descritos no Capítulo 4: cada passo foi implementado, executado e observado antes de o passo seguinte ser iniciado, em um padrão mais próximo de iterações curtas do que de uma sequência em cascata. Essa alternância está detalhada no Quadro 2, na Seção 3.4, que associa cada passo a uma decisão arquitetural explícita.

3.3 Arquitetura proposta e escolha tecnológica

Para simular a transição de um cenário fortemente acoplado para uma plataforma governada, a arquitetura da prova de conceito foi organizada em quatro camadas funcionais, descritas a seguir.

(a) Camada de dados, PostgreSQL com PL/pgSQL. O sistema legado foi representado por um banco PostgreSQL com cinco tabelas principais, *convenios*, *pacientes*, *medicos*, *consultas* e *eventos_faturamento*, e regras de negócio encapsuladas no próprio banco. A função *gerar_resumo_faturamento*, escrita em PL/pgSQL, realiza agregações e junções entre essas tabelas para calcular totais faturados, ressarcidos e pendentes por convênio em um

determinado período. A view `vw_paciente_detalhado` retorna dados do paciente incluindo o CPF em formato bruto, evidenciando o risco de exposição direta caso não haja uma camada intermediária de proteção.

(b) Camada de governança, API-First com OpenAPI e Policy-as-Code com Spectral. A definição dos endpoints foi feita por meio de um contrato OpenAPI 3.0.3, o arquivo `swagger.yaml`, seguindo a abordagem API-First. Esse contrato formaliza recursos, formatos de requisição e resposta, códigos HTTP e esquemas de segurança, funcionando como fonte única de verdade para o desenvolvimento. Para transformar diretrizes de governança em regras executáveis, adotou-se o Spectral como linter de contrato, com quatro regras customizadas codificadas no arquivo `.spectral.yaml`, descritas no Quadro 2.

Quadro 2 — Regras customizadas de Policy-as-Code (Spectral).

Regra	Severidade	Finalidade
<code>no-sensitive-data-in-path</code>	error	Bloqueia a presença de CPF, e-mail, RG ou senha em parâmetros de URL, conformidade com a LGPD.
<code>require-500-response</code>	error	Exige a documentação da resposta de erro interno, HTTP 500, em todos os endpoints.
<code>response-keys-camelcase</code>	warn	Impõe o padrão camelCase para as chaves JSON declaradas nas respostas.
<code>require-security-scheme</code>	warn	Exige a declaração de pelo menos um esquema de segurança no contrato.

Fonte: o autor (2026).

A integração entre governança e build foi formalizada por meio do script de build do backend, que encadeia a verificação do Spectral antes da compilação TypeScript. Contratos classificados com erros bloqueiam o build, impedindo que o código avance sem governança de contrato válida.

(c) Camada de plataforma, Node.js com Express e TypeScript. A camada intermediária foi implementada com Node.js, Express e TypeScript, organizada em subcamadas com responsabilidades bem delimitadas. Os controllers respondem exclusivamente pela leitura de parâmetros HTTP e pela montagem da resposta, sem conter lógica de negócio. Os services aplicam as regras de domínio, incluindo o mascaramento de CPF por meio da função `maskCPF`, que mantém visíveis apenas os dois dígitos verificadores e substitui os nove dígitos anteriores por asteriscos antes de qualquer dado sensível ser

transmitido ao consumidor. Essa estratégia de mascaramento parcial se classifica tecnicamente como pseudonimização, nos termos do art. 13 da LGPD: o dado original permanece no banco para fins administrativos, mas a camada governada limita a exposição ao mínimo necessário, sem comprometer a integridade referencial.

Os repositories isolam todo o acesso ao banco, encapsulando as chamadas às funções e views do PostgreSQL e impedindo que SQL vaze para as demais camadas, no sentido proposto por Fowler (2002) para a separação entre lógica de domínio e acesso a dados. Os middlewares cobrem autenticação JWT, autorização por papel, admin e user, verificação de payload com Zod e tratamento padronizado de erros.

O tratamento de erros foi padronizado em um middleware global que retorna sempre o formato { code, message, traceId }, mapeando os códigos HTTP 400, 401, 403, 404 e 500. O traceId é gerado como UUID por requisição, permitindo rastreabilidade sem expor detalhes internos do legado. A verificação de payload com Zod cobre os parâmetros dos endpoints existentes, rejeitando entradas malformadas com erro 400 no formato padronizado antes que cheguem à camada de serviço. Foram implementados também dois endpoints operacionais: GET /health, que confirma que o processo da API está em execução, liveness, e GET /ready, que executa um SELECT 1 no banco e retorna HTTP 200 quando a dependência está disponível ou HTTP 503, no formato de erro padronizado, quando não está, readiness.

(d) Camada de consumo, frontend em React com TypeScript. O frontend foi implementado em React com TypeScript e comunica-se exclusivamente com a API intermediária via HTTP. O fluxo de autenticação no cliente segue três etapas: o usuário submete credenciais na tela de login, que chama POST /auth/login, o token JWT recebido é armazenado e injetado automaticamente em todas as requisições subsequentes por meio de um interceptor do Axios, e as rotas protegidas verificam a existência do token, redirecionando para a tela de login na ausência ou expiração dele. O frontend não possui driver de conexão com o PostgreSQL nem conhecimento de stored procedures ou views do legado.

3.4 Processo de implementação e decisões arquiteturais

A implementação seguiu uma sequência de oito passos, ordenados por dependência técnica e relevância para os objetivos do trabalho. Cada passo foi guiado por uma decisão arquitetural explícita, registrada como justificativa para as escolhas feitas, material que conecta diretamente as decisões de engenharia aos objetivos acadêmicos. O Quadro 3 consolida esse mapeamento.

Quadro 3 — Passos de implementação e decisões arquiteturais.

Passo	O que foi implementado	Decisão arquitetural
1	Integração do Spectral ao build (lint:spectral antes do tsc).	O contrato OpenAPI é tratado como artefato obrigatório; build sem contrato válido é bloqueado.
2	Padronização do errorHandler com code, message e traceId; mapeamento de 400, 401, 403, 404 e 500.	Erros padronizados evitam o vazamento de detalhes internos do legado e facilitam a rastreabilidade.
3	Criação da camada repository; isolamento de todo o acesso SQL dos services.	Isolar o acesso a dados em repositories desacopla o domínio da persistência e viabiliza testes unitários sem banco real.
4	Verificação de payload com Zod; middleware validate aplicado por rota.	A verificação por schema garante o contrato em runtime e protege o legado de entradas malformadas.
5	Autenticação JWT; papéis admin e user; atualização do swagger.yaml com securitySchemes.	O JWT torna o controle de acesso explícito e auditável via contrato OpenAPI, mesmo em prova de conceito.
6	Configuração do Jest; testes unitários para maskCPF, service de faturamento e middlewares de autenticação.	Testes unitários isolam as regras de negócio da infraestrutura, verificando a camada anticorrupção sem dependência do banco.
7	Pipeline GitHub Actions com install, lint:spectral, test e build.	O pipeline garante governança contínua; nenhuma mudança avança sem contrato válido e testes passando.
8	Endpoint GET /ready com SELECT 1; retorno 503 padronizado em caso de falha.	Separar liveness de readiness permite detectar a indisponibilidade do banco sem derrubar a API.

Fonte: o autor (2026).

3.5 Critérios de verificação da prova de conceito

A verificação foi conduzida a partir de quatro critérios técnico-funcionais, de natureza binária, atendido ou não atendido, cada um associado a evidências de execução coletadas durante o projeto. Registra-se que esses critérios aferem a conformidade do artefato com propriedades arquiteturais desejadas, e

não medem intensidade, eficiência ou ganho relativo em relação a uma arquitetura sem governança equivalente.

O Critério 1, governança e bloqueio de não conformidades, verifica se o contrato OpenAPI é verificado automaticamente e se violações bloqueiam o build, tendo como evidência a saída do comando `npm run build`. O Critério 2, tratamento de dados sensíveis na camada anticorrupção, verifica se o consumidor nunca recebe o CPF em formato bruto, tendo como evidência a resposta do endpoint de paciente. O Critério 3, qualidade verificável por testes automatizados, verifica se as regras de negócio são cobertas por testes independentes de infraestrutura, tendo como evidência a saída do comando `npm test`. O Critério 4, desacoplamento técnico do consumidor, verifica se o frontend depende apenas da API e não possui acesso direto ao banco, tendo como evidência a ausência de drivers de banco no frontend e a inspeção das chamadas de rede.

3.6 Estrutura do repositório e reprodutibilidade

O projeto foi organizado em três módulos no repositório: `database/`, com o script `schema.sql` contendo DDL, DML, a função `gerar_resumo_faturamento` e a view `vw_paciente_detalhado`, `backend/`, com a implementação da API em Node.js e TypeScript, organizada nas camadas de `controllers`, `services`, `repositories`, `middlewares`, `schemas` e `config`, e `frontend/`, com a interface em React. O contrato OpenAPI, `swagger.yaml`, e o ruleset do Spectral, `.spectral.yaml`, estão na raiz do repositório, reforçando seu papel como artefatos centrais de governança. A reprodução do ambiente requer configurar o banco PostgreSQL e executar o `schema.sql`, iniciar o backend e iniciar o frontend, com as variáveis de ambiente descritas no arquivo de exemplo.

4 DESENVOLVIMENTO DA PROVA DE CONCEITO

Este capítulo descreve o desenvolvimento da prova de conceito que sustenta a hipótese central do trabalho: uma plataforma de APIs governada pode atuar como uma Camada Anticorrupção eficaz entre o sistema legado e

as aplicações modernas, reduzindo o acoplamento técnico e mitigando a exposição de dados sensíveis. O desenvolvimento foi conduzido em oito passos sequenciais, cada um guiado por uma decisão arquitetural explícita. A ordem não foi arbitrária: cada etapa cria a fundação técnica necessária para a seguinte, de modo que a sequência reflete tanto uma progressão de maturidade quanto uma estratégia de implementação rastreável, alinhada às atividades de projeto e desenvolvimento do ciclo DSR descrito na Seção 3.2. O cenário escolhido foi o de uma clínica médica fictícia, justificado pela densidade de regras de negócio envolvidas e pelo requisito de conformidade com a LGPD no tratamento de dados pessoais sensíveis.

4.1 Camada de dados legada: regras e riscos estruturais

A camada de persistência foi implementada com o SGBD PostgreSQL, simulando um ambiente corporativo no qual regras de negócio permanecem acopladas ao banco. O script `schema.sql` contém a definição completa do esquema: cinco tabelas relacionais, `convenios`, `pacientes`, `medicos`, `consultas` e `eventos_faturamento`, dados de exemplo e dois objetos centrais para a prova de conceito. O primeiro é a função `gerar_resumo_faturamento`, escrita em PL/pgSQL, que realiza agregações e junções entre as tabelas para calcular totais faturados, ressarcidos e pendentes por convênio em um determinado período. Esse tipo de lógica, consolidada no banco, representa alto acoplamento: qualquer consumidor que precise desse dado precisaria conhecer a estrutura interna do banco para obtê-lo de forma consistente. O segundo é a view `vw_paciente_detalhado`, que retorna dados agregados do paciente incluindo o CPF em formato bruto, um ponto crítico de vulnerabilidade caso clientes externos acessem o banco diretamente, configurando potencial não conformidade com a LGPD.

Código-fonte 1 — `database/schema.sql` (assinatura da função de faturamento).

```
-- Definição da função legada (PL/pgSQL), encapsulada no repository
CREATE OR REPLACE FUNCTION gerar_resumo_faturamento(
    p_mes INTEGER,
    p_ano INTEGER
)
RETURNS TABLE (
    total_consultas BIGINT,
    total_faturado NUMERIC,
    total_ressarcido NUMERIC,
```

```
total_pendente NUMERIC,  
consultas_por_convenio JSONB  
) AS $$  
BEGIN  
    -- agregações e joins entre consultas, pacientes e eventos_faturamento  
    RETURN QUERY ... ;  
END;  
$$ LANGUAGE plpgsql;
```

Fonte: o autor (2026).

4.2 Passo 1 — Integração do contrato OpenAPI ao build

O primeiro passo tratou de tornar o contrato OpenAPI parte obrigatória do processo de build. Antes dessa intervenção, o arquivo `swagger.yaml` existia no repositório, mas a verificação via Spectral era executada de forma independente da compilação TypeScript, o que permitia que o código avançasse mesmo com um contrato fora de conformidade. A mudança foi pontual: o script build do `package.json` do backend passou a encadear a execução do Spectral antes do compilador. Se o Spectral identifica erros no contrato, o processo falha e a compilação não ocorre.

Código-fonte 2 — `backend/package.json` (script de build encadeado).

```
{  
  "scripts": {  
    "lint:spectral": "spectral lint ../swagger.yaml --ruleset ../.spectral.yaml",  
    "build": "npm run lint:spectral && tsc"  
  }  
}
```

Fonte: o autor (2026).

O output obtido na execução confirma o comportamento esperado: o Spectral reportou sete warnings e zero erros no estado atual do contrato. Como o ruleset está configurado para bloquear apenas erros, e não warnings, o build prosseguiu para a compilação TypeScript, que concluiu com sucesso. Qualquer violação classificada como erro, como a exposição de CPF em parâmetros de URL, interromperia o processo nesse ponto. Os sete warnings reportados se referem a aspectos de completude documental do contrato, descritos em detalhe na seção de resultados, e não a falhas funcionais.

Código-fonte 3 — saída real do build (evidência coletada).

```
> tcc-plataforma-medica-backend@1.0.0 build  
> npm run lint:spectral && tsc  
  
> tcc-plataforma-medica-backend@1.0.0 lint:spectral  
> spectral lint ../swagger.yaml --ruleset ../.spectral.yaml  
  
[X] 7 problems (0 errors, 7 warnings, 0 infos, 0 hints)
```


Fonte: o autor (2026).

4.3 Passo 2 — Padronização do tratamento de erros

O segundo passo formalizou o tratamento de erros da API. O middleware errorHandler já existia, mas seu formato de resposta não era padronizado, o que criava inconsistência entre endpoints e dificultava a rastreabilidade de falhas, algo especialmente relevante em um contexto com dados sensíveis, no qual detalhes internos do legado não devem vaziar para o consumidor. O middleware foi reescrito para retornar sempre o mesmo envelope de erro, independentemente da origem da falha. Um identificador único de rastreamento, o traceId, gerado por UUID a cada requisição com erro, permite correlacionar logs sem expor informações internas.

Código-fonte 4 — backend/src/middlewares/errorHandler.ts (formato padronizado).

```
export class AppError extends Error {
  statusCode: number;
  code: string;
  constructor(message: string, statusCode: number, code: string) {
    super(message);
    this.statusCode = statusCode;
    this.code = code;
  }
}

// Resposta sempre neste formato:
res.status(statusCode).json({ code, message, traceId });
```

Fonte: o autor (2026).

O mapeamento de códigos HTTP cobre os cenários mais comuns: 400 para erros de validação de entrada, 401 para requisições sem autenticação válida, 403 para tentativas de acesso sem o papel necessário, 404 para recursos inexistentes e 500 para erros internos não tratados. Esse mapeamento está documentado no contrato OpenAPI, garantindo consistência entre o que o código retorna e o que o contrato declara.

4.4 Passo 3 — Criação da camada repository

O terceiro passo introduziu a camada repository, isolando completamente o acesso ao banco de dados das demais camadas da aplicação. Antes dessa refatoração, os services continham queries SQL diretamente, o que acoplava as regras de negócio ao mecanismo de persistência e tornava os testes unitários dependentes de uma conexão real

com o banco, exatamente o tipo de acoplamento que Fowler (2002) associa a sistemas difíceis de testar. Foram criados dois repositories correspondentes aos dois recursos principais: o de faturamento, que encapsula a chamada à função PL/pgSQL de resumo, e o de pacientes, que encapsula a consulta à view vw_paciente_detalhado. A partir desse passo, os services passaram a receber os dados já carregados pelos repositories e a aplicar apenas transformações de domínio, como o mascaramento de CPF, sem qualquer conhecimento do SQL subjacente.

4.5 Passo 4 — Validação de payload com Zod

O quarto passo introduziu a verificação de payload em runtime por meio da biblioteca Zod. Antes dessa implementação, os parâmetros de entrada chegavam aos services sem verificação de tipo ou formato, o que expunha o banco legado a consultas com dados malformados e tornava os erros de entrada difíceis de distinguir de erros internos. Foram criados schemas Zod para cada endpoint: o de faturamento verifica os parâmetros de query mes, inteiro entre 1 e 12, e ano, inteiro entre 2020 e 2030, o de pacientes verifica o parâmetro de rota id como inteiro positivo. Um middleware genérico validate foi implementado para aplicar esses schemas antes que a requisição chegue ao controller, em caso de falha, encaminha um erro 400 no formato padronizado definido no Passo 2.

Código-fonte 5 — backend/src/middlewares/validate.ts (middleware genérico de verificação).

```
type ValidationSchema = { params?: ZodTypeAny; query?: ZodTypeAny };

export const validate = (schemas: ValidationSchema) => {
  return (req, _res, next) => {
    if (schemas.params) {
      const r = schemas.params.safeParse(req.params);
      if (!r.success) {
        return next(new AppError(msg, 400, 'BAD_REQUEST'));
      }
    }
    if (schemas.query) {
      const r = schemas.query.safeParse(req.query);
      if (!r.success) {
        return next(new AppError(msg, 400, 'BAD_REQUEST'));
      }
    }
    next();
  };
};
```

Fonte: o autor (2026).

4.6 Passo 5 — Autenticação JWT e controle de acesso por papel

O quinto passo implementou a autenticação e a autorização de ponta a ponta, fechando a lacuna identificada no protótipo original, no qual o contrato declarava `bearerAuth`, mas o backend não o aplicava e o frontend não enviava token algum. A implementação cobre os dois lados da integração: o backend emite e verifica tokens JWT, com senhas armazenadas em hash `bcrypt`, fator de custo 10, o frontend possui tela de login, persistência de token e um interceptor de requisição que injeta o cabeçalho de autorização automaticamente. Foi criado o endpoint `POST /auth/login`, que recebe credenciais e retorna um token JWT assinado. Para o contexto desta prova de conceito, as credenciais são verificadas contra usuários predefinidos com papéis `admin` e `user`. O middleware `authenticateJWT` verifica a validade do token e retorna 401 em caso de ausência ou invalidade, o middleware `authorizeRoles` verifica se o papel contido no token satisfaz os requisitos da rota e retorna 403 em caso contrário.

Código-fonte 6 — `swagger.yaml` (declaração do esquema de autenticação).

```
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
      description: Token JWT com role em claim para autorização por endpoint.
```

Fonte: o autor (2026).

4.7 Passo 6 — Testes unitários com Jest

O sexto passo configurou o ambiente de testes e implementou cobertura unitária para as regras de negócio mais críticas. Antes dessa etapa, o script de teste retornava sucesso sem executar nenhum caso, o que tornava o pipeline de integração contínua ineficaz. Foram configurados o Jest e o `ts-jest`, e criadas três suítes de teste. A primeira cobre a função `maskCPF` com casos normais e de borda, a segunda cobre o service de faturamento com mock do repository, verificando a transformação dos dados sem depender de conexão real, a terceira cobre os middlewares de autenticação, com cenários de token ausente, token inválido e papel insuficiente. A execução confirma três suítes e oito testes, todos passando.

Código-fonte 7 — saída real do npm test (evidência coletada).

```
PASS tests/services/faturamento.service.test.ts
PASS tests/middlewares/auth.middleware.test.ts
PASS tests/utils/cpfMask.test.ts

Test Suites: 3 passed, 3 total
Tests:      8 passed, 8 total
Snapshots:  0 total
```

Fonte: o autor (2026).

4.8 Passo 7 — Pipeline de integração contínua (GitHub Actions)

O sétimo passo formalizou a governança contínua por meio de um pipeline de integração contínua configurado no GitHub Actions. O arquivo de workflow define etapas executadas sequencialmente a cada push ou pull request: checkout do código, configuração do Node.js, instalação de dependências com npm ci, verificação do contrato OpenAPI com Spectral, execução dos testes unitários e build final com compilação TypeScript. A ordem das etapas reflete a estratégia de falha antecipada: o contrato é verificado antes dos testes, e os testes são executados antes do build, de modo que nenhuma mudança avança sem que o contrato esteja em conformidade, os testes passem e a compilação seja bem-sucedida. Qualquer falha em uma etapa interrompe o pipeline e impede o merge.

Código-fonte 8 — .github/workflows/ci.yml (pipeline de integração contínua).

```
name: CI
on:
  push:
    branches: [ '*' ]
  pull_request:
jobs:
  backend-ci:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: backend } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm run lint:spectral
      - run: npm test
      - run: npm run build
```

Fonte: o autor (2026).

4.9 Passo 8 — Health checks: liveness e readiness

O oitavo e último passo completou a observabilidade básica da API com a separação entre dois tipos de health check. O endpoint GET /health confirma

apenas que o processo Node.js está em execução, liveness. O endpoint GET /ready foi implementado executando um SELECT 1 no banco PostgreSQL: se a consulta retorna com sucesso, responde com HTTP 200, se falha, por indisponibilidade do banco, erro de rede ou credenciais inválidas, responde com HTTP 503, sempre no formato de erro padronizado, incluindo o traceId. Vale registrar que esses dois endpoints são operacionais e ficam fora do contrato de negócio declarado no swagger.yaml, por decisão de projeto: o contrato formaliza os recursos de domínio, autenticação, faturamento e pacientes, enquanto os health checks atendem a necessidades de infraestrutura.

4.10 Visão consolidada da arquitetura final

Ao final dos oito passos, a arquitetura atingiu um nível de maturidade superior ao do protótipo original. O Quadro 4 consolida os componentes implementados, os arquivos correspondentes e o papel de cada um.

Quadro 4 — Componentes da arquitetura final.

Componente	Arquivo(s)	Papel na arquitetura
Contrato OpenAPI	swagger.yaml	Fonte única de verdade; define rotas, schemas, segurança e erros.
Ruleset Spectral	.spectral.yaml	Policy-as-Code; bloqueia contratos fora de conformidade no build.
Repository de Faturamento	faturamento.repository.ts	Encapsula a chamada à função PL/pgSQL do legado.
Repository de Pacientes	pacientes.repository.ts	Encapsula a consulta à view com CPF bruto.
Service de Pacientes	pacientes.service.ts	Aplica maskCPF antes de montar o DTO de resposta.
Middlewares	auth.ts, validate.ts, errorHandler.ts	Autenticação e autorização JWT, verificação Zod e padronização de erros.
Pipeline CI	ci.yml	Executa lint, testes e build a cada push; falha em qualquer etapa.
Health checks	GET /health, GET /ready	Confirmam liveness do processo e readiness do banco.

Fonte: o autor (2026).

A arquitetura resultante mostra que a abordagem de Engenharia de Plataforma não só endereça o problema técnico do acoplamento, mas o faz de

forma verificável e rastreável. Cada decisão implementada pode ser demonstrada com evidências concretas: saídas de terminal, respostas de endpoints, logs do pipeline e cobertura de testes.

5 RESULTADOS E DISCUSSÃO

Este capítulo apresenta os resultados obtidos na execução da prova de conceito e os discute à luz do referencial teórico. A análise está organizada em torno dos quatro critérios definidos na metodologia. Todos foram verificados a partir de evidências de execução reproduzíveis, coletadas em ambiente local com o backend em operação e o banco PostgreSQL populado. De modo geral, a prova de conceito demonstrou a viabilidade da abordagem: a combinação de API-First, Policy-as-Code e Camada Anticorrupção mostrou-se capaz de isolar o consumidor moderno do sistema legado, ao mesmo tempo em que introduziu controles verificáveis de segurança e qualidade.

5.1 Critério 1 — Governança de contrato e bloqueio de não conformidades

O primeiro critério avalia se o contrato OpenAPI é verificado automaticamente e se violações bloqueiam o avanço do build. A integração do Spectral ao script de build estabeleceu uma barreira formal entre o código e o contrato. A saída coletada confirma que o processo encadeia corretamente a verificação antes da compilação, reportando sete warnings e zero erros no estado atual da especificação, com a compilação concluindo em seguida. O comportamento de bloqueio é assegurado pelo ruleset, que classifica como erros as violações mais críticas, como a presença de dados sensíveis em parâmetros de URL, impedindo que contratos com essas características avancem.

Os sete warnings remanescentes, agora identificados nominalmente, referem-se exclusivamente a aspectos de completude documental do contrato, e não a falhas de segurança ou de funcionamento: ausência de objeto de licença (info-license e license-url), preenchimento incompleto do objeto de

contato (contact-properties), ausência de operationId em três operações, e ausência de descrição em uma operação. São avisos não bloqueantes, plenamente endereçáveis em iteração futura, e sua distinção em relação aos erros bloqueantes é, em si, um resultado relevante: evidencia a diferença prática entre o que é desejável e o que é obrigatório na governança automatizada. Esse resultado se conecta com a recomendação de Chandramouli (2022) de tornar políticas explícitas e verificáveis no ciclo de desenvolvimento: a abordagem Code-First costuma postergar a verificação do contrato para revisões manuais tardias, ao passo que a abordagem adotada inverte essa lógica, tornando o contrato um artefato de primeira classe no processo de entrega.

Código-fonte 9 — detalhamento dos sete warnings do Spectral (evidência coletada).

```
9:6   warning  info-license      Info object must have "license" object.
9:6   warning  license-url      License object must include "url".
24:11 warning  contact-properties Contact object must have name, url and email.
183:10 warning operation-description Operation description must be present.
183:10 warning operation-operationId Operation must have operationId.
225:9  warning operation-operationId Operation must have operationId.
320:9  warning operation-operationId Operation must have operationId.
```

```
[X] 7 problems (0 errors, 7 warnings, 0 infos, 0 hints)
```

Fonte: o autor (2026).

5.2 Critério 2 — Tratamento de dados sensíveis na camada anticorrupção

O segundo critério avalia se o consumidor recebe o CPF sempre mascarado, nunca em formato bruto, resultado de maior relevância para a conformidade com a LGPD. Com o backend em execução e autenticado por um token JWT válido, a requisição GET /api/v1/pacientes/1 retornou o objeto com o campo cpfMascarado no padrão `***.***.***-01`, com apenas os dois dígitos verificadores visíveis. O dado completo existe na view `vw_paciente_detalhado` no banco, mas é interceptado pela função `maskCPF` no service antes de qualquer serialização para o consumidor. A inspeção da resposta confirma que o CPF em formato bruto não trafega em momento algum entre o backend e o frontend.

Código-fonte 10 — resposta real do endpoint de paciente (CPF pseudonimizado).

```
GET /api/v1/pacientes/1 (Authorization: Bearer <token>) -> HTTP 200
{
  "id": 1,
  "nomeCompleto": "Maria Silva Santos",
  "cpfMascarado": "***.***.***-01",
```

```
"dataNascimento": "1985-03-15T03:00:00.000Z",  
"telefone": "11987654321",  
"email": "maria.silva@email.com",  
"convenioId": 1,  
"totalConsultas": 2,  
"totalGasto": 650  
}
```

Fonte: o autor (2026).

Do ponto de vista arquitetural, esse resultado valida o papel da Camada Anticorrupção descrito por Evans (2003): a ACL não apenas traduz modelos, mas também aplica políticas de proteção que seriam impossíveis de garantir se o consumidor tivesse acesso direto ao banco. A LGPD, em seu art. 46, estabelece que os agentes de tratamento devem adotar medidas técnicas e administrativas aptas a proteger os dados pessoais de acessos não autorizados. O mascaramento implementado na camada de plataforma representa exatamente esse tipo de medida técnica, operando de forma centralizada e independente da ação individual do desenvolvedor. Cabe a ressalva técnica já apresentada no referencial: trata-se de pseudonimização, art. 13, e não de anonimização, art. 12, pois o controlador mantém a capacidade de reidentificação a partir do dado original preservado no banco.

Como verificação complementar do mesmo critério, a requisição ao endpoint de paciente sem token de autenticação foi corretamente rejeitada com HTTP 401, retornando o envelope de erro padronizado, o que confirma que a proteção do recurso opera em runtime e não apenas no contrato.

Código-fonte 11 — teste negativo de autenticação (evidência coletada).

```
GET /api/v1/pacientes/1 (sem token) -> HTTP 401  
{  
  "code": "UNAUTHORIZED",  
  "message": "Token ausente ou inválido",  
  "traceId": "39172124-802f-47e1-abf2-609ee233f981"  
}
```

Fonte: o autor (2026).

5.3 Critério 3 — Qualidade verificável por testes automatizados

O terceiro critério avalia se as regras de negócio são cobertas por testes independentes de infraestrutura. Três suítes foram implementadas, cobrindo a função maskCPF com casos normais e de borda, o service de faturamento com mock do repository e os middlewares de autenticação com cenários de token ausente, inválido e papel insuficiente. A execução do comando npm test

confirmou oito testes em três suítes, todos passando, condição que se manteve estável após as modificações finais do projeto, incluindo a correção do carregamento de variáveis de ambiente. O pipeline de integração contínua verifica essa condição a cada push, impedindo que regressões avancem sem detecção.

A decisão de escrever testes unitários sem dependência de banco real, viabilizada pelo isolamento da camada repository no Passo 3, mostra na prática o valor arquitetural dessa separação. Fowler (2018) argumenta que testes dependentes de infraestrutura externa tendem a ser mais lentos e frágeis, ao passo que testes unitários isolados são rápidos, determinísticos e podem rodar em qualquer contexto. O fato de o pipeline de integração contínua conseguir executar os testes sem uma instância real do PostgreSQL é um resultado direto dessa decisão arquitetural.

5.4 Critério 4 — Desacoplamento técnico do consumidor em relação ao legado

O quarto critério avalia se o frontend depende apenas da API e não possui acesso direto ao banco. A verificação estrutural confirmou a ausência de qualquer driver de banco entre as dependências do frontend: o package.json declara apenas react, react-dom, react-router-dom e axios, sem pg, postgres, knex, sequelize ou typeorm. As ocorrências do termo PostgreSQL encontradas no frontend se referem exclusivamente a comentários de documentação e a uma string de interface, não a código de conexão. O interceptor do Axios garante que todas as chamadas incluam o token JWT, e a inspeção das requisições confirma que o frontend comunica-se apenas com a API intermediária, sem comunicação direta com o banco.

Do ponto de vista conceitual, o resultado é coerente com o padrão ACL: o frontend conhece apenas os contratos definidos no swagger.yaml, e não as stored procedures, views ou a estrutura interna do PostgreSQL. Essa separação é precisamente o que Evans (2003) descreve como o principal benefício da camada anticorrupção: o consumidor moderno pode evoluir independentemente do legado, e o legado pode ser modificado sem quebrar o contrato público, desde que a ACL absorva as diferenças.

5.5 Síntese dos resultados e discussão

O Quadro 5 sintetiza o resultado de cada critério, o status de verificação e a evidência associada.

Quadro 5 — Síntese da verificação dos critérios.

Critério	Status	Evidência
Governança de contrato (Policy-as-Code)	Atendido	Saída do build: 0 errors, 7 warnings, documentais, não bloqueantes.
Mascaramento de CPF (LGPD)	Atendido	Resposta HTTP 200 com cpfMascarado ***.***.***-01; acesso sem token rejeitado com 401.
Testes automatizados (Jest)	Atendido	3 suítes / 8 testes passando, estáveis após as modificações finais.
Desacoplamento frontend/banco	Atendido	Ausência de driver de banco no frontend; tráfego exclusivo para a API.

Fonte: o autor (2026).

Um achado adicional merece registro, por ilustrar com precisão os limites da governança baseada apenas em contrato. A resposta do endpoint de faturamento retornou o objeto consolidado corretamente, porém a estrutura aninhada `consultasPorConvenio` apresentou a chave `convenio_id` em `snake_case`, em desacordo com o padrão `camelCase` adotado no restante da resposta. Essa inconsistência não foi capturada pela regra `response-keys-camelcase` do Spectral porque tal regra inspeciona o contrato declarado, e não o payload gerado em runtime, o campo, por sua vez, se origina diretamente do `jsonb_build_object` da função PL/pgSQL legada. O caso demonstra empiricamente que a governança de contrato é necessária, mas não suficiente: dados que nascem crus no banco podem contornar a padronização se a camada de tradução não os normalizar explicitamente. Trata-se de um ponto de melhoria concreto para a evolução da plataforma e de um argumento a favor de reforçar a normalização na camada de service.

Código-fonte 12 — resposta do endpoint de faturamento, com chave aninhada em `snake_case`.

```
GET /api/v1/faturamento/resumo?mes=2&ano=2025 -> HTTP 200
{
  "totalConsultas": 4,
  "totalFaturado": 1530,
  "totalRessarcido": 150,
  "totalPendente": 380,
  "consultasPorConvenio": [
    { "quantidade": 1, "convenio_id": 0, "valor_total": 500 },
```

```
{ "quantidade": 3, "convenio_id": 1, "valor_total": 1030 }  
]  
}
```

Fonte: o autor (2026).

Comparado ao estado inicial do protótipo, no qual o contrato existia mas não era integrado ao build, a autenticação estava declarada mas não implementada, e os testes não executavam nada, o progresso é substancial. O sistema passou de um protótipo com governança aspiracional para uma plataforma com governança verificável, autenticação funcional, verificação de payload em runtime e cobertura de testes nas regras de negócio críticas. Esse resultado se alinha ao que a literatura de Engenharia de Plataforma descreve como a transição de APIs ad hoc para produtos digitais governados: não se trata de adicionar complexidade, mas de tornar explícitas e automatizadas as restrições que antes dependiam de disciplina individual.

6 CONCLUSÃO

Este trabalho partiu de uma pergunta prática e recorrente em organizações que operam sistemas legados: como permitir que aplicações modernas consumam dados e regras de negócio consolidados no banco sem expor a estrutura interna, sem comprometer a segurança e sem criar novas dívidas técnicas no processo? A resposta proposta foi a construção de uma plataforma de APIs governada, sustentada por três pilares complementares: a Camada Anticorrupção como padrão de isolamento arquitetural, a abordagem API-First com OpenAPI como mecanismo de contrato, e a Governança como Código com Spectral como forma de tornar políticas verificáveis e automatizadas.

A prova de conceito desenvolvida no contexto de uma clínica médica fictícia demonstrou que essa combinação é viável e implementável com tecnologias amplamente disponíveis. Os oito passos de implementação percorridos, da integração do contrato ao build até a separação entre liveness e readiness, constituem uma sequência replicável que qualquer equipe com

contexto semelhante poderia adotar de forma incremental, sem necessidade de reescrever o sistema legado.

6.1 Resposta à questão de pesquisa

A questão de pesquisa indagava de que maneira uma camada intermediária baseada em API-First e Policy-as-Code poderia reduzir o acoplamento entre clientes modernos e um legado com regras no banco, elevando a padronização do contrato e o controle sobre a evolução da API. As evidências coletadas sustentam uma resposta afirmativa quanto à viabilidade. O mascaramento de CPF na camada de serviço, confirmado em requisição HTTP real, demonstrou que dados sensíveis podem ser protegidos por arquitetura, e não apenas por processos. A integração do Spectral ao build demonstrou que políticas de qualidade podem ser impostas mecanicamente. A separação em camadas de controller, service, repository e middleware demonstrou que é possível organizar a API de modo que cada responsabilidade seja testável e auditável de forma independente. E a implementação de JWT com papéis demonstrou que o controle de acesso pode ser formalizado no próprio contrato OpenAPI, eliminando a lacuna entre o que é declarado e o que é praticado.

Cabe delimitar o alcance dessa resposta. As evidências sustentam que a arquitetura proposta é viável e que os mecanismos de governança operam conforme projetado. A redução de acoplamento foi demonstrada em termos estruturais, ausência de driver de banco no consumidor, isolamento do SQL na camada repository, contrato como única interface conhecida pelo cliente, e não quantificada por meio de métricas comparativas. Uma afirmação de superioridade da abordagem exigiria baseline, grupo de controle e medição, elementos deliberadamente fora do escopo desta prova de conceito.

6.2 Limitações do trabalho

As limitações a seguir delimitam o alcance das conclusões e orientam a agenda de pesquisa apresentada na Seção 6.4.

Validade externa. A prova de conceito foi construída e avaliada em um cenário controlado e fictício, concebido para reproduzir as características

estruturais de um sistema legado de clínica médica. Embora o cenário replique o padrão de regras de negócio encapsuladas em PL/pgSQL e de exposição de identificadores sensíveis, ele não incorpora a escala, a heterogeneidade nem a dívida técnica acumulada de um sistema em operação real. A generalização dos resultados para ambientes produtivos permanece, portanto, uma hipótese a ser verificada.

Ausência de validação com profissionais. A arquitetura proposta não foi submetida à apreciação de arquitetos de software ou de equipes que operam sistemas legados em produção. Uma proposta que se pretende replicável se beneficiaria de avaliação por especialistas quanto à sua aplicabilidade, aos custos de adoção e aos pontos de atrito em contextos reais.

Ausência de baseline comparativo. O trabalho não estabelece um cenário de controle, uma integração equivalente sem governança, contra o qual medir o ganho da abordagem. Os critérios verificam conformidade, não intensidade, o que sustenta a tese de viabilidade, mas não uma afirmação de superioridade mensurada.

Escopo do sistema de identidade. O mecanismo JWT foi implementado com usuários predefinidos e senhas em hash bcrypt, cobrindo o ciclo completo de autenticação, mas sem persistência de usuários no banco, renovação de tokens, refresh token, ou revogação de sessões. Em produção, seria necessário integrar um provedor de identidade externo com essas capacidades.

Cobertura de testes. As três suítes cobrem as regras de negócio mais críticas, porém não há testes de integração com banco real nem testes ponta a ponta cobrindo o fluxo do frontend até o PostgreSQL. Também não foram executados testes com usuários ou consumidores externos.

Ausência de avaliação de desempenho. A prova de conceito não inclui métricas de latência, testes de carga, análise de escalabilidade, rastreamento distribuído ou observação de comportamento sob volume.

Escopo da análise de conformidade. O tratamento da LGPD é consistente no plano arquitetural, a distinção entre pseudonimização, art. 13, e anonimização, art. 12, é observada, e o mascaramento é implementado como

medida técnica nos termos do art. 46. Contudo, o trabalho não realiza avaliação institucional, jurídica ou organizacional mais ampla, tampouco endereça aspectos como base legal do tratamento, registro de operações ou governança de dados no nível da organização.

Registra-se, por fim, que durante a verificação foi identificado e corrigido um defeito de inicialização relacionado à ordem de carregamento das variáveis de ambiente, que impedia a subida do servidor. A correção foi aplicada e as evidências de runtime foram coletadas a partir do sistema já corrigido.

6.3 Contribuições do trabalho

Além de demonstrar a viabilidade da abordagem proposta, este trabalho produz três contribuições. A primeira é a demonstração prática de que a Engenharia de Plataforma não é exclusividade de grandes organizações com equipes dedicadas: a prova de conceito foi construída por um único desenvolvedor, com tecnologias de código aberto, sugerindo que os princípios de governança de API são acessíveis mesmo em contextos menores. A segunda é o conjunto de decisões arquiteturais documentadas ao longo dos oito passos, cada uma registrada com uma justificativa explícita que conecta a escolha técnica ao objetivo de negócio ou de conformidade que ela serve, modelo de documentação que torna a arquitetura mais explicável e auditável. A terceira é a evidência de que a conformidade com a LGPD pode ser tratada como uma propriedade arquitetural verificável, e não apenas como um conjunto de políticas organizacionais, conforme ilustrado pelo mascaramento de CPF incorporado ao desenho do sistema.

6.4 Sugestões para trabalhos futuros

A partir das limitações identificadas, sugerem-se seis direções de continuidade. A primeira é a validação da proposta junto a arquitetos de software e equipes que operam sistemas legados em produção, por meio de entrevistas ou avaliação por especialistas. A segunda é a implementação de testes de integração com banco real por meio de contêineres efêmeros, com ferramentas como Testcontainers, verificando o comportamento completo do fluxo desde o repository até a resposta HTTP. A terceira é a evolução do

mecanismo de autenticação, substituindo a verificação baseada em tokens JWT locais por uma infraestrutura de federação de identidades, primeiro passo prático para a adoção de uma arquitetura de confiança zero. A quarta é o amadurecimento da observabilidade, com logs estruturados correlacionados por `traceld`, métricas exportadas para ferramentas como o Prometheus e rastreamento distribuído com OpenTelemetry. A quinta é a aplicação da mesma abordagem a um cenário com múltiplos sistemas legados e múltiplos consumidores, introduzindo versionamento de API, gestão do ciclo de vida de contratos e estratégias de depreciação controlada.

A sexta direção é a avaliação comparativa da abordagem. A ideia é construir um cenário de controle, uma integração equivalente sem governança, e medir contra ele o ganho real da plataforma proposta, acompanhando essa comparação de testes ponta a ponta que percorram todo o fluxo, do consumidor no frontend até o PostgreSQL, e de métricas de desempenho sob carga, como latência e comportamento sob volume. É justamente esse conjunto de medições que permitiria transformar a tese de viabilidade demonstrada aqui em uma afirmação quantificada de redução de acoplamento e de ganho operacional, algo que esta prova de conceito, por opção, não se propôs a fazer.

6.5 Consideração final

A epígrafe escolhida para este trabalho, de Martin Fowler, afirma que bons programadores escrevem código que humanos entendam. Essa ideia, aplicada ao contexto de APIs e sistemas legados, pode ser ampliada: boas arquiteturas produzem sistemas que humanos conseguem auditar, evoluir e, sobretudo, confiar. O que este trabalho procurou demonstrar é que governança, segurança e qualidade não são propriedades que se adicionam a um sistema depois de pronto, elas são consequências de decisões arquiteturais tomadas cedo, documentadas com clareza e verificadas de forma contínua. Sistemas legados não precisam ser reescritos para se tornarem modernos, eles precisam de uma camada que os proteja, os traduza e os governe. É isso que uma plataforma de APIs bem construída pode fazer.

REFERÊNCIAS

ALMEIDA, Nathalia R. de; CAMPOS, Gabriel N.; MORAES, Fernando R. de; AFFONSO, Frank J. Modernization of Legacy Systems to Microservice Architecture: a tertiary study. In: PROCEEDINGS OF THE 26TH INTERNATIONAL CONFERENCE ON ENTERPRISE INFORMATION SYSTEMS (ICEIS 2024), v. 2. [S. l.]: SciTePress, 2024. p. 581-592. DOI: 10.5220/0012633300003690.

ASSUNÇÃO, Wesley K. G.; MARCHEZAN, Luciano; ARKOH, Lawrence; EGYED, Alexander; RAMLER, Rudolf. Contemporary Software Modernization: strategies, driving forces, and research opportunities. ACM Transactions on Software Engineering and Methodology, v. 34, n. 5, 2025. DOI: 10.1145/3708527.

BENNETT, Keith H.; RAJLICH, Václav T. Software maintenance and evolution: a roadmap. In: PROCEEDINGS OF THE CONFERENCE ON THE FUTURE OF SOFTWARE ENGINEERING (ICSE 2000). Limerick: ACM, 2000. p. 73-87. DOI: 10.1145/336512.336534.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União, Brasília, DF, 15 ago. 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 25 jun. 2026.

CHANDRAMOULI, Ramaswamy. Implementation of DevSecOps for a Microservices-based Application with Service Mesh. NIST Special Publication 800-204C. Gaithersburg: National Institute of Standards and Technology, 2022. DOI: 10.6028/NIST.SP.800-204C.

EVANS, Eric. Domain-Driven Design: tackling complexity in the heart of software. Boston: Addison-Wesley, 2003.

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

FOWLER, Martin. StranglerFigApplication. martinfowler.com, 2004. Disponível em: <https://martinfowler.com/bliki/StranglerFigApplication.html>. Acesso em: 25 jun. 2026.

FOWLER, Martin. Refactoring: improving the design of existing code. 2. ed. Boston: Addison-Wesley, 2018.

HEVNER, Alan R.; MARCH, Salvatore T.; PARK, Jinsoo; RAM, Sudha. Design Science in Information Systems Research. MIS Quarterly, v. 28, n. 1, p. 75-105, 2004. DOI: 10.2307/25148625.

OPENAPI INITIATIVE. OpenAPI Specification 3.0.3. 2021. Disponível em: <https://spec.openapis.org/oas/v3.0.3>. Acesso em: 25 jun. 2026.

OWASP. API9:2023 Improper Inventory Management. OWASP API Security Top 10, 2023. Disponível em: <https://owasp.org/API-Security/editions/2023/en/0xa9-improper-inventory-management/>. Acesso em: 17 jul. 2026.

PEFFERS, Ken; TUUNANEN, Tuure; ROTHENBERGER, Marcus A.; CHATTERJEE, Samir. A Design Science Research Methodology for Information Systems Research. Journal of Management Information Systems, v. 24, n. 3, p. 45-77, 2007. DOI: 10.2753/MIS0742-122240302.

RICHARDSON, Chris. Microservices Patterns. Shelter Island: Manning, 2018.

SEACORD, Robert C.; PLAKOSH, Daniel; LEWIS, Grace A. Modernizing Legacy Systems: software technologies, engineering processes and business practices. Boston: Addison-Wesley, 2003.

SKELTON, Matthew; PAIS, Manuel. Team Topologies: organizing business and technology teams for fast flow. Portland: IT Revolution Press, 2019.

STOPLIGHT. Spectral: a flexible JSON/YAML linter for OpenAPI. 2021. Disponível em: <https://stoplight.io/open-source/spectral>. Acesso em: 25 jun. 2026.

WOLFART, Daniele; ASSUNÇÃO, Wesley K. G.; SILVA, Ivonei F. da; DOMINGOS, Diogo C. P.; SCHMEING, Ederson; VILLAÇA, Guilherme L. D.; PAZA, Diogo do N. Modernizing Legacy Systems with Microservices: a roadmap. In: EVALUATION AND ASSESSMENT IN SOFTWARE ENGINEERING (EASE 2021). New York: ACM, 2021. p. 149-159. DOI: 10.1145/3463274.3463334.